



**UNIVERSITÀ
DEGLI STUDI
DI TRIESTE**

Università degli Studi di Trieste

Dipartimento di Ingegneria e Architettura

Corso di laurea magistrale in Ingegneria Elettronica e Informatica
[curriculum Informatica]

Sunto della tesi di laurea magistrale

Video Acquisition and Processing on Intel Cyclone V SoC

*Hardware Implementation with Embedded Linux
and Bare-Metal Software Architectures*

Laureando
Tiziano Ceccarini

Relatore
Prof. Stefano Marsi

Anno accademico 2025–2026

1 Introduzione

La crescente diffusione di sensori, telecamere e dispositivi connessi rende sempre meno conveniente e, in alcuni casi, impraticabile trasferire integralmente tutti i dati grezzi a un server remoto. In applicazioni quali l'ispezione visiva di una linea produttiva, la guida di un braccio robotico o il monitoraggio ambientale, è spesso necessario ottenere una risposta in tempi molto brevi, ossia a bassa latenza, e poterla ottenere anche quando la connessione di rete non è disponibile o non è affidabile. A questa esigenza risponde l'*edge computing*, paradigma nel quale almeno una parte dell'elaborazione viene eseguita direttamente sul dispositivo che genera i dati, o nelle sue immediate vicinanze.

L'edge computing non è quindi un ambito soltanto sperimentale. Secondo una previsione pubblicata da IDC nel 2026, la spesa mondiale associata all'edge computing, pari a 265 miliardi di dollari nel 2025, è attesa a raggiungere 450 miliardi nel 2029.¹ La sua rilevanza strategica è riconosciuta anche dall'Unione europea, che nel programma Digital Decade ha fissato per il 2030 l'obiettivo di 10.000 nodi edge climaticamente neutri e altamente sicuri.²

Un *edge device* è quindi un sistema che non si limita a raccogliere e trasmettere informazioni, ma esegue localmente almeno l'elaborazione necessaria a produrre un risultato utilizzabile. Una telecamera collocata in un parcheggio, ad esempio, può stabilire quali stalli siano liberi e comunicare il loro numero, anziché inviare in permanenza l'intero flusso video. Il cloud non viene così eliminato, ma impiegato per ciò che richiede risorse centralizzate, quali l'archiviazione dei dati, l'analisi su larga scala o il coordinamento di più dispositivi.

2 Possibili architetture per gli edge device

Gli edge device possono essere realizzati con architetture differenti. Una soluzione consiste nell'impiegare un microcontrollore (MCU) o un microprocessore (MPU) all'interno di un sistema embedded, cioè un sistema di elaborazione integrato in un dispositivo per svolgere funzioni specifiche. Il processore esegue il software e mette a disposizione periferiche hardware già definite dal costruttore del chip. Quando il compito richiede un'interfaccia o una logica non disponibile fra le periferiche standard, oppure un'elaborazione fortemente specifica dell'applicazione, al processore può essere affiancata una FPGA (*Field-Programmable Gate Array*). A differenza delle soluzioni basate sulle sole MCU o MPU, una FPGA consente al progettista di definire una logica hardware personalizzata, cioè un circuito costruito per lo specifico compito. Le operazioni più regolari possono così essere accelerate direttamente dall'hardware, riducendo la latenza di elaborazione e sollevando il processore da calcoli e trasferimenti non necessari. Questa ripartizione del lavoro contribuisce a migliorare l'efficienza energetica del sistema. Il processore resta così maggiormente disponibile per le operazioni più adatte alla sua natura, quali le decisioni, la comunicazione e le logiche che cambiano nel tempo.

¹International Data Corporation, *Edge Computing Global Spending to Grow at 15%, reaching \$450 Billion by 2029*, 26 febbraio 2026, <https://www.idc.com/resource-center/press-releases/edgecomputingforecast/>.

²European Commission, *Cloud computing*, <https://digital-strategy.ec.europa.eu/en/policies/cloud-computing>.

Il sistema realizzato in questa tesi è stato sviluppato sulla scheda Terasic DE1-SoC. Il suo componente principale, un Intel Cyclone V SoC, implementa un moderno approccio alla realizzazione di dispositivi edge, che consiste nell'integrare nello stesso chip una FPGA e un sistema di elaborazione basato su un MPU dual-core con architettura ARM Cortex-A9. Intel chiama *Hard Processor System* (HPS) la parte non riconfigurabile del chip, che comprende la MPU, il controllore della memoria DDR3 e altre periferiche, quali i controllori USB e l'interfaccia di rete. Collegamenti dedicati permettono alla MPU di interfacciarsi con la logica realizzata nella FPGA, che dal punto di vista del software può essere trattata come una periferica.

3 Obiettivi della tesi

L'obiettivo della tesi è realizzare un sistema completo di acquisizione ed elaborazione video. L'acquisizione delle immagini dalla telecamera assegnata, dotata di un'interfaccia non standard, è affidata alla logica FPGA; l'elaborazione è invece eseguita via software dalla MPU ARM integrata nel dispositivo. Il punto di partenza è quindi una logica programmabile non configurata, nella quale non esistono né un'interfaccia per la telecamera né il percorso necessario per portare le immagini alla memoria del sistema.

4 Catena di acquisizione video

4.1 Collegamento con la telecamera

La realizzazione ha richiesto di intervenire anche sul collegamento fisico fra telecamera e scheda. Alla frequenza con cui il sensore trasmette i dati, i cavetti jumper inizialmente impiegati degradavano il segnale fino a renderne inaffidabile la lettura da parte della FPGA. Per questo sono stati progettati e realizzati due circuiti stampati adattatori, dotati di buffer di linea attivi, che rendono stabile il collegamento fra telecamera e scheda di sviluppo.

Risolto il collegamento elettrico, è stato necessario costruire nella FPGA l'interfaccia digitale del sensore. La telecamera impiegata, basata sul sensore OmniVision OV7670, trasmette i pixel su un bus parallelo a otto bit, scandito da un proprio segnale di temporizzazione, detto clock, e delimitato da segnali di riga e di fotogramma; non esiste un meccanismo con cui la FPGA possa chiederle di rallentare o attendere. La logica di acquisizione deve quindi essere pronta a ricevere ogni campione al suo arrivo, riconoscere l'inizio e la fine di ciascun fotogramma e consegnarlo ordinatamente alla catena di trasferimento.

4.2 Trasferimento delle immagini in memoria

Il passo successivo è portare i dati acquisiti nella memoria DDR3, dove saranno elaborati. Un requisito centrale del progetto è che questo trasferimento avvenga senza impiegare la MPU nella copia dei singoli dati, lasciando disponibili i suoi cicli di calcolo per l'elaborazione delle immagini.

La soluzione adottata è un DMA (*Direct Memory Access*), ossia un controllore hardware

capace di trasferire autonomamente blocchi di dati. Alla MPU è sufficiente comunicare l'indirizzo della memoria di destinazione e la dimensione del trasferimento; da quel momento il DMA prende in carico il flusso in ricezione e lo scrive in memoria, senza che la MPU debba eseguire istruzioni di copia per ogni byte in arrivo. Al termine del trasferimento, il controllore ne notifica il completamento alla MPU mediante un interrupt, un segnale che richiama l'attenzione del processore affinché esegua la relativa procedura di gestione.

Nel Cyclone V SoC non è disponibile, nella parte MPU, un DMA già predisposto per questo trasferimento. Intel mette però a disposizione un componente hardware configurabile, denominato *Modular Scatter-Gather DMA* (mSGDMA), che in questa tesi è stato integrato nella logica della FPGA e collegato al controllore della memoria DDR3 dell'HPS attraverso connessioni dedicate.

4.3 Sincronizzazione e fotogrammi incompleti

Per collegare l'interfaccia di acquisizione al DMA occorre inoltre adattare i rispettivi tempi di funzionamento. I dati arrivano secondo il clock della telecamera, mentre il DMA opera con il clock del sistema e può subire brevi pause dovute alla contesa per l'accesso alla memoria DDR3. Fra i due sono state quindi inserite memorie temporanee, dette buffer, che consentono il passaggio dei dati fra parti del circuito governate da clock differenti (domini di clock) e accumulano i dati in attesa che il trasferimento possa proseguire.

Se gli stadi successivi non riescono a ricevere altri dati, lo comunicano attraverso un segnale di *backpressure*. Quando i buffer non sono più sufficienti a compensare l'arresto, questo segnale si propaga fino all'interfaccia di acquisizione. Poiché la telecamera non può essere fermata, la logica abbandona il fotogramma in corso e, quando il trasferimento riprende, ne segnala la chiusura anticipata al DMA. Il software riceve anche il numero effettivo di byte trasferiti e può pertanto riconoscere e scartare l'immagine incompleta, senza trattarla come un fotogramma valido.

5 Sistemi realizzati e applicazioni dimostrative

La catena di acquisizione descritta costituisce la base comune di due sistemi sviluppati nella tesi, denominati *FPGAlix* e *FPGAsteel*. Entrambi affidano alla FPGA l'acquisizione e il trasferimento delle immagini in memoria, mantenendo l'elaborazione sulla MPU, ma adottano due differenti ambienti software. FPGAlix impiega il sistema operativo Linux, che mette a disposizione servizi e strumenti per lo sviluppo delle applicazioni; FPGAsteel esegue invece il programma direttamente sul processore, senza sistema operativo, secondo un approccio detto *bare metal*.

5.1 FPGAlix: acquisizione in ambiente Linux

In FPGAlix è stato sviluppato un kernel driver, ossia un componente software che permette a Linux di controllare la catena di acquisizione realizzata nella FPGA e renderla accessibile alle applicazioni attraverso Video4Linux2 (V4L2), l'interfaccia standard di Linux

per le periferiche video. La telecamera appare quindi alle applicazioni come un normale dispositivo video: un programma può acquisire i fotogrammi senza dover conoscere l'organizzazione hardware e software del percorso verso la memoria. Su questa base, utilizzando librerie open source disponibili per Linux, è stata realizzata un'applicazione dimostrativa, denominata *vision_core*, che acquisisce il flusso video, applica via software sulla MPU una catena di filtri configurabile a caldo, cioè modificabile senza interrompere l'elaborazione del flusso video, e trasmette il risultato attraverso l'interfaccia Ethernet della scheda. La dimostrazione coinvolge quindi l'intero percorso dei dati, dal sensore alla FPGA, dalla memoria al processore e infine alla rete.

5.2 FPGAsteeel: elaborazione e visualizzazione in locale

In FPGAsteeel, l'assenza di un sistema operativo comporta la necessità di predisporre esplicitamente il controllo delle periferiche. Per trasmettere il video come in FPGAlinx sarebbe stato necessario sviluppare un driver Ethernet e implementare i protocolli di comunicazione richiesti, affrontando un lavoro ampio e complesso, non pertinente agli obiettivi di questa tesi. Per poter comunque osservare il flusso video risultante dall'elaborazione, è stata utilizzata l'uscita VGA presente sulla DE1-SoC, collegabile direttamente a un monitor. Alla catena di acquisizione è stato quindi affiancato nella FPGA un secondo percorso DMA, che compie il tragitto inverso: legge dalla memoria DDR3 i fotogrammi elaborati e li invia al controllore VGA.

Rispetto a FPGAlinx, in ciascuno dei due DMA di FPGAsteeel, quello della telecamera e quello dell'uscita VGA, è stato aggiunto un modulo interno opzionale denominato *prefetcher*. Questo modulo preleva direttamente da una memoria dedicata l'indirizzo del buffer e le altre informazioni necessarie a eseguire il trasferimento dei dati. Al termine di ciascun trasferimento, il prefetcher ne segnala il completamento alla MPU mediante un interrupt; la MPU seleziona quindi il buffer successivo e riarma il DMA. Il processore interviene così nella gestione dei fotogrammi, ma non deve occuparsi del trasferimento dei singoli pixel. Si ottiene un percorso video interamente locale, nel quale la FPGA gestisce l'ingresso e l'uscita dei dati, mentre la MPU rimane disponibile per l'elaborazione delle immagini.

Lo sviluppo di FPGAsteeel è stato accompagnato da una serie di applicazioni dimostrative organizzate in ordine progressivo. Ciascuna introduce e verifica una nuova funzionalità, riutilizzando quelle già collaudate: l'avvio del sistema e il controllo delle periferiche, la configurazione della telecamera, il trasferimento delle immagini verso l'uscita VGA, l'ottimizzazione dell'esecuzione software, l'avvio del secondo core ARM e la successiva ripartizione dell'elaborazione fra i due core. Questo procedimento consente di verificare separatamente i diversi componenti e di individuare eventuali problemi prima di integrarli nel sistema completo; permette inoltre al lettore meno esperto di acquisire gradualmente familiarità con il lavoro svolto.

La serie culmina nell'applicazione più complessa, che utilizza entrambi i core ARM per individuare un oggetto chiaro su sfondo scuro. Il programma converte il fotogramma in scala di grigi, applica una soglia per separare l'oggetto dallo sfondo e, mediante operazioni morfologiche, elimina i disturbi isolati e ne determina il contorno. Il software sovrappone quindi al video mostrato sul monitor VGA il contorno dell'oggetto, il riquadro che lo racchiude e una croce che ne indica il baricentro. L'applicazione verifica così l'intera

catena locale di acquisizione, elaborazione e visualizzazione e sottopone la MPU a un carico misurabile.

5.3 Ottimizzazione e prestazioni della MPU

Per valutare le prestazioni della MPU sono stati impiegati tre strumenti di accelerazione offerti dai core ARM. Il primo è la cache dati, che conserva vicino al processore i dati utilizzati di recente o collocati in indirizzi vicini, sfruttando il principio di località; il secondo è la cache delle istruzioni, che svolge una funzione analoga sul codice in esecuzione. Il terzo è costituito dalle istruzioni vettoriali, denominate NEON da ARM, che consentono di applicare contemporaneamente la stessa operazione a più pixel.

La cache dati rimane attiva in tutte le prove. L'applicazione finale è stata compilata in tre varianti che eseguono lo stesso algoritmo: la versione di riferimento non utilizza le istruzioni vettoriali NEON e mantiene disattivata la cache delle istruzioni; la seconda abilita tale cache; nella terza il codice è stato inoltre riscritto esplicitamente per utilizzare le istruzioni NEON. Il tempo necessario a elaborare un fotogramma passa rispettivamente da 142 a 47 e infine a 17 millisecondi. La versione completa risulta quindi circa 8,35 volte più veloce di quella di riferimento.

6 Considerazioni finali e sviluppi futuri

Partendo da una FPGA non configurata, il lavoro ha costruito l'intero percorso necessario ad acquisire un flusso video da una telecamera con interfaccia non standard e a trasferirne i fotogrammi nella memoria DDR3. La logica FPGA e il DMA si occupano dello spostamento dei singoli byte, mentre la MPU controlla il trasferimento a livello di fotogramma ed elabora le immagini quando queste sono disponibili in memoria. Su questa infrastruttura sono stati realizzati due sistemi, FPGAlix e FPGAsteele, che rendono disponibile la medesima architettura di acquisizione rispettivamente in ambiente Linux e *bare metal*. Le applicazioni dimostrative sviluppate sui due sistemi hanno infine verificato il funzionamento congiunto di sensore, logica FPGA, memoria, processore e interfacce di uscita.

Nelle applicazioni dimostrative, le operazioni sulle immagini sono state mantenute intenzionalmente sulla MPU per sottoporre a un carico significativo il processore e l'accesso alla memoria DDR3. È stato così possibile osservare il comportamento della catena di acquisizione mentre il software era impegnato nell'analisi dei fotogrammi. Le misure ottenute quantificano i benefici delle ottimizzazioni introdotte e forniscono un riferimento per sviluppi successivi, nei quali le diverse funzioni potranno essere ripartite fra MPU e logica FPGA.

Le caratteristiche della piattaforma realizzata ne consentono l'impiego come base per la progettazione di un edge device dedicato alla visione e al controllo. Le immagini possono essere acquisite ed elaborate localmente; verso l'esterno può quindi essere trasmesso il video già elaborato oppure, introducendo un'applicazione specifica, soltanto l'informazione ricavata, come una misura, un conteggio o una segnalazione. Il risultato dell'elaborazione può anche comandare direttamente uno o più attuatori: in una linea produttiva, per esempio, il dispositivo potrebbe individuare un componente difettoso me-

dianete ispezione visiva e attivare un meccanismo incaricato di scartarlo. L'elaborazione in prossimità del sensore riduce così il lavoro demandato ai sistemi centrali o al cloud e la quantità di dati trasferita attraverso la rete. Il dispositivo può inoltre continuare a prendere decisioni quando la connessione non è disponibile o non è affidabile, evitando i ritardi dovuti alla comunicazione con un sistema remoto. FPGAlix dimostra il funzionamento della piattaforma in un ambiente connesso attraverso Ethernet, mentre FPGAsteeel mostra che, nella configurazione realizzata, l'intera catena di acquisizione, elaborazione e visualizzazione opera esclusivamente in locale, con uscita VGA e senza i ritardi e la variabilità temporale dovuti all'alternanza fra i programmi gestita dal sistema operativo (schedulazione).

Un ulteriore sviluppo consiste nel trasferire alla FPGA operazioni regolari eseguite nello stesso modo per ogni pixel, come la conversione di formato, la sogliatura, i filtri di convoluzione e le elaborazioni morfologiche, eseguendole direttamente sul flusso video in arrivo. Ciò ridurrebbe il carico della MPU e la latenza di elaborazione, lasciando al software le decisioni di livello più elevato e le funzioni che devono essere modificate più frequentemente. Le configurazioni FPGAlix e FPGAsteeel utilizzano rispettivamente circa il 15% e il 18% della logica programmabile disponibile e nessuna delle due impiega alcuno degli 87 blocchi hardware del Cyclone V SoC dedicati alle operazioni aritmetiche e all'elaborazione dei segnali. Rimane quindi un margine di risorse che potrebbe essere utilizzato per realizzare questi acceleratori.

La documentazione prodotta e i sorgenti hardware e software resi disponibili nei repository dedicati descrivono puntualmente i passaggi necessari a riprodurre entrambi i sistemi. Questa impostazione permette a chi proseguirà il lavoro di comprenderne le scelte progettuali, riprodurre il funzionamento e utilizzare la piattaforma come base per applicazioni personalizzate ed estensioni future, senza dover ricostruire da principio l'intera catena di acquisizione ed elaborazione.